Rapport de Travaux Pratiques

Atelier 3 — Smart Contract « Mini Réseau Social » en Solidity

12 novembre 2026

1 Objectifs et contexte

Ce rapport présente la mise en œuvre d'un smart contract décentralisé permettant :
— La publication de messages courts par des utilisateurs identifiés par leur adresse Ethereum
— Le stockage persistant des publications dans un tableau dynamique
— La consultation publique des messages et de leurs auteurs respectifs

Stack technique : Solidity `0.8.20`, Remix IDE, Remix VM (environnement local simulé).

2 Implémentation et déploiement local

2.1 Structure du contrat Solidity

Le contrat `MiniSocial` définit une structure `Post` composée de deux champs :
— `message` : contenu textuel (string)
— `author` : adresse Ethereum de l'émetteur (address)
Les publications sont stockées dans un tableau dynamique public `posts[]`.

Fonction	Type	Description
`publishPost(string)`	Transaction	Ajoute un nouveau post au tableau
`getPost(uint)`	View	Retourne le message et l'auteur à l'index spécifié
`getTotalPosts()`	View	Retourne le nombre total de publications

TABLE 1 – Fonctions implémentées dans le contrat

2.2 Déploiement dans Remix VM

Le contrat a été compilé puis déployé avec succès dans l'environnement **Remix VM (Osaka)**.

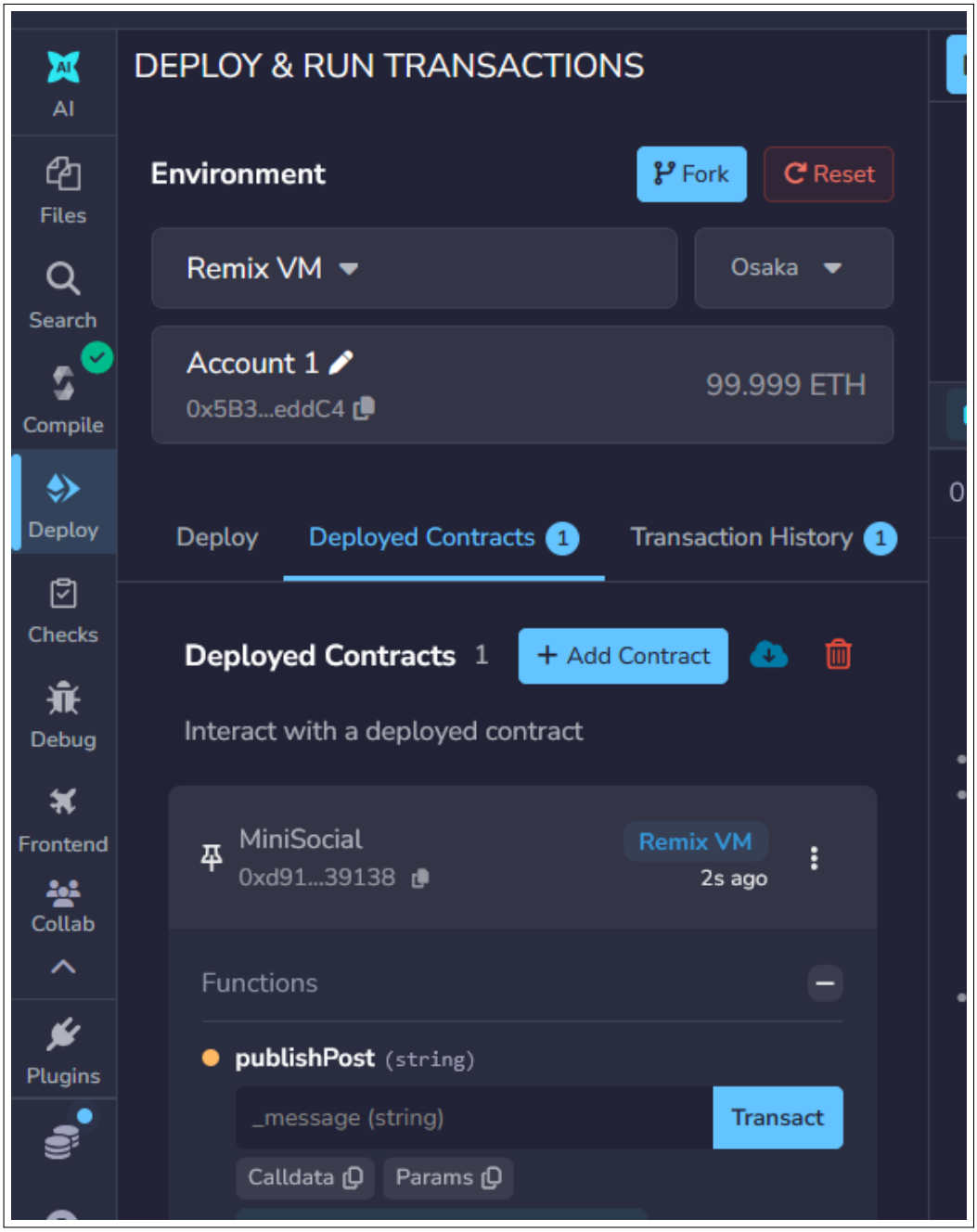


FIGURE 1 – Instance MiniSocial visible dans « Deployed Contracts »

Interprétation : La présence du contrat dans la liste des contrats déployés et la transaction de création confirmée attestent que l'instance est opérationnelle dans l'environnement local. Les ETH utilisés sont des fonds simulés sans valeur réelle.

3 Scénarios de test

3.1 Premier scénario : publication avec Account 1

Adresse : `0x5b363c6d29719c4da335a3d5e8f972465253ddc4`

3.1.1 Exécution de `publishPost`

Une première tentative a généré une erreur d'encodage (fourniture de deux arguments au lieu d'un). Après correction, la transaction a été confirmée :

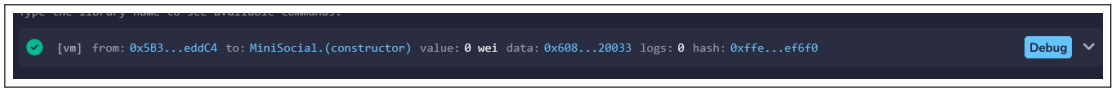


FIGURE 2 – Transaction `publishPost` confirmée après correction du paramétrage

Message publié :

« Bonjour, ceci est mon premier message sur MiniSocial ! »

3.1.2 Vérification de l'état

Lecture du total :

```
getTotalPosts() → 1
```

Lecture du post index 0 :

```
getPost(0) →
  message: "Bonjour, ceci est mon premier message sur MiniSocial !"
  author: 0x5b363c6d29719c4da335a3d5e8f972465253ddc4
```

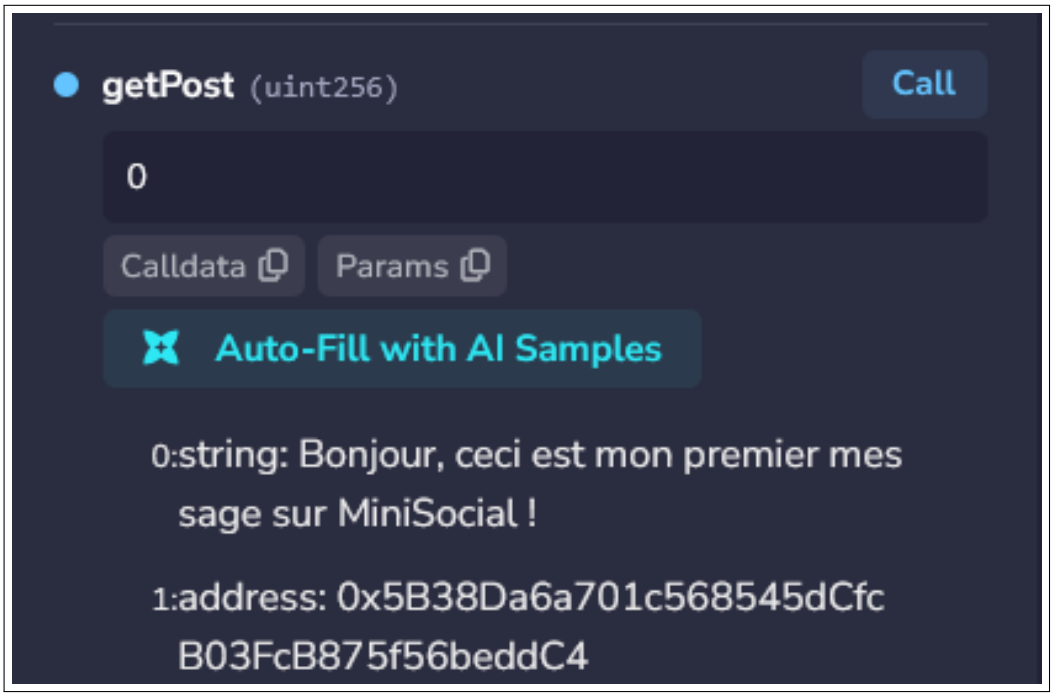


FIGURE 3 – Résultat de `getPost(0)` affichant message et adresse

Interprétation : L'index 0 correspond au premier élément inséré. L'adresse retournée correspond exactement à celle d'Account 1, confirmant l'association automatique du message à son émetteur via `msg.sender`.

3.2 Second scénario : publication avec Account 2

Adresse : `0xab85dc7743b10111e54e7b395181f995446c5cb2`

3.2.1 Changement de contexte d'exécution

Sélection d'Account 2 dans l'interface Remix (solde : 100.000 ETH).



FIGURE 4 – Changement de compte actif

3.2.2 Publication et vérification

Message publié :

« Bonjour, je publie depuis le deuxième compte ! »

Résultat de `getPost(1)` :

```
message: "Bonjour, je publie depuis le deuxième compte !"
author: 0xab85dc7743b10111e54e7b395181f995446c5cb2
```



FIGURE 5 – Résultat de `getPost(1)`

Interprétation : L'auteur du second message diffère du premier. Le contrat identifie nativement les utilisateurs par leur adresse Ethereum, sans nécessité de système d'authentification ou de nom d'utilisateur supplémentaire.

4 Vérification finale et synthèse

4.1 État final du contrat

`getTotalPosts() → 2`



FIGURE 6 – Comptage final : `getTotalPosts()` retourne 2

4.2 Tableau récapitulatif des tests

Test	Résultat observé	Conclusion
Déploiement dans Remix VM	Transaction de création confirmée	Réussi
Publication compte 1	`publishPost` confirmé	Réussi
Consultation index 0	Message 1 + adresse Account 1	Conforme
Publication compte 2	`publishPost` confirmé	Réussi
Consultation index 1	Message 2 + adresse Account 2	Conforme
Comptage final	`getTotalPosts() = 2`	Conforme

TABLE 2 – Synthèse des tests effectués

5 Analyse et limites

5.1 Analyse technique

Architecture des données : Le tableau dynamique `Post[]` stocke les publications dans l'ordre chronologique d'insertion (FIFO). Chaque élément est immuable une fois écrit.

Modèle de coût :

— `publishPost` modifie l'état du contrat (écriture en storage) → nécessite une transaction coûteuse en gas

— `getPost` et `getTotalPosts` sont des fonctions `view` (lecture seule) → gratuites, exécutées localement

Gestion de l'identité : L'utilisation de `msg.sender` garantit l'authenticité de l'auteur sans mécanisme de login, exploitant la cryptographie asymétrique native d'Ethereum.

5.2 Limites identifiées et travail restant

Limite	Impact	Solution envisageable
Environnement Remix VM uniquement	Pas de test sur réseau public	Déploiement sur Sepolia (testnet)
Publications immuables	Pas de correction/suppression possible	Ajouter fonctions `editPost`/`deletePost` avec vérification d'autorisation
Absence de modération	Contenu non filtrable	Système de signalement ou whitelist d'auteurs
Pas de persistance externe	Données perdues si redéploiement	Intégration IPFS/Arweave pour le contenu
Dépôt GitHub manquant	Pas de traçabilité du code	Initialiser repo public avec le code source

TABLE 3 – Limites et pistes d'amélioration

6 Conclusion générale

L'Atelier 3 a permis de développer et de valider un smart contract **MiniSocial** fonctionnel en Solidity. Les exigences fonctionnelles ont été intégralement couvertes :

Structure `Post` avec message et auteur
Stockage dans un tableau dynamique
Fonctions `publishPost`, `getPost` et `getTotalPosts` opérationnelles
Identification des utilisateurs par adresse Ethereum

Les tests effectués depuis deux comptes distincts ont confirmé le bon fonctionnement du contrat : deux publications enregistrées, relues avec succès, avec attribution correcte des auteurs et comptage final exact.

Portée de la validation : Cette démonstration reste limitée à l'environnement local Remix VM. Elle valide la logique métier et l'absence d'erreurs de programmation, mais ne constitue pas une preuve de fonctionnement en conditions réelles de réseau (latence, coûts réels, concurrence).

Prochaines étapes :

1. Déploiement sur le testnet Sepolia pour tester sur infrastructure publique
2. Vérification du contrat sur Etherscan
3. Publication du code source sur GitHub avec documentation
4. (Optionnel) Développement d'une interface web (Web3.js/Ethers.js) pour interagir avec le contrat sans passer par Remix